SGSR-2: Measured-Cost Pareto Allocation of Bit-Width and Group-Size for On-Device LLM Quantization

Matthias Raviotta
Independent Research
raviottamatthias@gmail.com

Abstract

We present **SGSR-2**, a post-training quantization method that jointly allocates per-block bit-width and group-size for large language models using *measured* quantization cost rather than heuristic proxies. For each transformer block and each configuration (b, k) with bits $b \in \{3, 4, 5, 6\}$ and group size $k \in \{32, 64, 128\}$, we measure the KL divergence of output logits under reversible fake-quantization of that block alone. A validated additivity assumption then reduces budget-constrained allocation to an exact per-block Lagrangian rule, yielding the complete 3–5 bit/w Pareto frontier from a single overnight profiling run on consumer hardware (Apple M1 Pro, 32 GB). Under a unified accounting protocol (on-disk size over total parameters, full Wikitext-2 sliding-window perplexity), SGSR-2 strictly dominates uniform MLX quantization on both TinyLlama-1.1B and Qwen2.5-7B—e.g. $+18.7\%$ PPL at 3.62 bit/w versus $+36.0\%$ at 3.93 bit/w for the best uniform setting on Qwen—and matches or outperforms the hand-tuned llama.cpp K-quants frontier at ≥ 4.4 bit/w, while losing to `Q3_K_M` below 4 bit/w, a gap we attribute to storage-format richness rather than allocation quality. We also report a negative result that corrects our earlier work (SGSR v1): under controlled baselines, Shannon-entropy sensitivity ranking is statistically indistinguishable from random ranking on both models, and the previously reported gains are explained by a SmoothQuant confound. All code, cost tables, and results are released; the method is gradient-free and runs entirely on-device.

This paper supersedes SGSR v1 (Zenodo, July 2026). Section 5 reports the controlled experiments that overturned v1's central claim.

1 Introduction

Deploying large language models on consumer hardware requires aggressive weight compression. Post-training quantization (PTQ) is the dominant approach [Frantar et al., 2022, Dettmers et al., 2022]. Two design axes govern the quality/size trade-off of group-wise PTQ: the *bit-width* of each weight and the *group size* over which quantization scales are shared. Smaller groups reduce quantization error but pay metadata overhead; in MLX's affine format each group of k weights carries a bf16 scale and bias, i.e. $32/k$ extra bits per weight. In practice both axes are set uniformly across all layers.

Transformer blocks, however, differ widely in their sensitivity to quantization error. Our earlier work (SGSR v1) attempted to exploit this by ranking layers with Shannon entropy and redistributing group sizes by rank. When we later added the controls that v1 lacked—the same SmoothQuant pre-processing on the uniform baseline, random and inverted rankings, and a uniform fine-group

Preprint. Under review.

baseline—the entropy signal vanished entirely (Section 5). The lesson is general: *heuristic proxies for sensitivity must be validated against randomized controls before they are used to allocate anything.*

SGSR-2 replaces the proxy with measurement. We fake-quantize one block at a time, measure the resulting KL divergence on the output logits over a fixed calibration set, and record a cost table $e_l(c)$ for every block l and configuration $c = (b, k)$. If total degradation is approximately additive across blocks—an assumption we test explicitly and quantify—then minimizing predicted cost under an effective-bits budget decomposes into an exact per-block rule $c_l(\lambda) = \arg\min_c \, e_l(c) + \lambda \, p_l \, \beta(c)$, where p_l is the block's parameter count and $\beta(c)$ its effective bits per weight. Sweeping the multiplier λ traces the entire Pareto frontier at negligible cost: one profiling run, every budget.

Our contributions are:

1. **A measured, gradient-free cost model** for joint (bit-width, group-size) allocation: per-block KL cost tables computed with reversible fake-quantization and a streaming top-K logit approximation, entirely on-device.
2. **An explicit additivity gate**: we validate that predicted (summed) costs track measured full-plan costs (ratio 0.74–1.11, rank order preserved) before trusting the allocator.
3. **An exact Lagrangian allocator** over the joint configuration space, producing the full 3–5 bit/w frontier from one cost table.
4. **A unified evaluation protocol**: all methods—ours, uniform MLX, and llama.cpp K-quants—are compared by on-disk size over total parameters, with full-test-set sliding-window perplexity and bootstrap confidence intervals. This corrects inconsistent accounting in v1.
5. **A documented negative result**: entropy ranking is indistinguishable from random ranking on both models once the SmoothQuant confound is removed.

2 Method

2.1 Measured per-block cost tables

Let a model have blocks $l = 1, \ldots, L$ (transformer decoder layers; all weight matrices within a block share one configuration). The configuration space is $\mathcal{C} = \{3, 4, 5, 6\} \times \{32, 64, 128\}$ (12 configurations); 2-bit is excluded due to catastrophic degradation and 8-bit is never optimal below a 5 bit/w average budget.

Reversible fake-quantization. For block l and configuration c, we replace every 2-D weight W in the block with $\mathrm{dequant}(\mathrm{quant}(W; c))$, keeping computation in BF16. The original tensors are retained and restored bit-exactly after measurement (guaranteed also on exceptions). Weight selection is fixed across all configurations (alignment to the largest group size), so every entry of the cost table measures the identical weight set—costs are comparable across configurations by construction.

Streaming KL cost. Storing full baseline logits for the calibration set is infeasible (sequence length $\times$ vocabulary $\approx$ 150k). We snapshot, per token, the top-64 baseline log-probabilities plus a residual tail mass, and compute

$$e_l(c) \;=\; \mathbb{E}_{\text{tokens}}\Big[\sum_{i \in \text{top-64}} p_i \log \frac{p_i}{q_i} \;+\; m_p \log \frac{m_p}{m_q}\Big] \;\geq\; 0, \tag{1}$$

where p is the baseline distribution, q the distribution under fake-quantization of block l, and m the respective tail masses. The calibration set is fixed and reproducible: 32 sequences of 512 tokens from Wikitext-2 train, seed 42. Profiling all $L \times |\mathcal{C}|$ entries takes one overnight run for a 7B model on M1 Pro; the table is cached and checkpointed per block, so interrupted runs resume.

2.2 Additivity gate

The allocator assumes total cost is approximately the sum of per-block costs. Before using it, we test this: for three plans spanning the budget range we apply the *full plan* in fake-quantization and

Table 1: Additivity gate on TinyLlama-1.1B. Predicted $= \sum_l e_l(c_l)$; measured = KL of the full plan applied jointly. Rank order across the three plans is preserved.

Plan (eff. bit/w)	Predicted	Measured	Ratio
3.25	0.2478	0.3351	0.74
3.81	0.1203	0.1389	0.87
4.61	0.0419	0.0378	1.11

compare the measured KL against the predicted sum (Table 1). Acceptance requires preserved rank order and prediction/measurement ratios within $[0.5, 1.5]$. The gate passed on first evaluation; had it failed, the protocol prescribes conditional re-measurement of the highest-cost blocks.

2.3 Exact Lagrangian allocation

Define effective bits $\beta(b, k) = b + 32/k$ (bf16 scale and bias per group) and let p_l be block l's parameter count. The budget-constrained problem

$$\min_{\{c_l\}} \sum_l e_l(c_l) \quad \text{s.t.} \quad \frac{\sum_l p_l \, \beta(c_l)}{\sum_l p_l} \;\le\; B \tag{2}$$

decomposes, under additivity, into the per-block rule

$$c_l(\lambda) \;=\; \arg\min_{c \in \mathcal{C}} \; e_l(c) + \lambda \, p_l \, \beta(c). \tag{3}$$

Sweeping λ over an adaptive range (anchored to each block's $\Delta e/(p_l \, \Delta\beta)$ turning points, with λ=0 included to guarantee the max-bits extreme) traces the frontier; deduplication by assignment yields the discrete Pareto set. For any requested budget we select the feasible point of minimum predicted cost. Fallback bits for modules outside the per-block plan (embeddings) track the parameter-weighted mean of the assigned block bits, keeping comparisons with uniform baselines fair. The allocator is pure post-processing: new budgets cost milliseconds once the table exists.

3 Experimental setup

Models and hardware. TinyLlama-1.1B-Chat-v1.0 [Zhang et al., 2024] and Qwen2.5-7B-Instruct [Team, 2024]; Apple M1 Pro, 32 GB unified memory, MLX [Apple Machine Learning Research, 2023] 0.31, BF16. No external accelerator at any stage.

Evaluation protocol. Perplexity on the *full* Wikitext-2 test set [Merity et al., 2017], sliding window 2048 with stride 512 (each token scored exactly once with at least 1536 tokens of context), 95% bootstrap confidence intervals over per-token negative log-likelihoods. SmoothQuant [Xiao et al., 2022] (α=0.5) is applied to *all* MLX-quantized variants including uniform baselines, removing the confound of v1.

Unified bits accounting. For every method, bits per weight = on-disk model size $\times 8$ divided by *total* parameter count. This penalizes all metadata (scales, biases, embeddings) identically across methods and matches how K-quants sizes are conventionally reported. We flag that v1 mixed two accounting conventions in one table; all numbers here use the unified rule.

Baselines. (i) Uniform MLX affine quantization at $(b, k) \in \{3, 4, 5\} \times \{32, 64, 128\}$ with SmoothQuant; (ii) llama.cpp K-quants [Gerganov and contributors, 2023] `Q3_K_M`, `Q4_K_M`, `Q5_K_M`, evaluated with `llama-perplexity` (context 2048) on the byte-identical test text, reported as ΔPPL relative to the f16 GGUF baseline of the same runtime, which controls for protocol differences between runtimes.

4 Results

SGSR-2 versus uniform MLX quantization. SGSR-2 strictly dominates the uniform frontier in the 3.4–4.5 bit/w region on both models. On Qwen, SGSR-2 reaches $+18.7\%$ at 3.62 bit/w where

Table 2: Main results under unified accounting (bit/w = disk size × 8 / total parameters). ΔPPL% relative to each runtime's own full-precision baseline; full Wikitext-2 test, sliding window. Uniform rows include SmoothQuant. The Qwen uniform sweep at 4–5 bits was truncated by an interrupted run (Section 7); measured points only.

		TinyLlama 1.1B		**Qwen2.5-7B**	
Family	**Setting**	bit/w	ΔPPL%	bit/w	ΔPPL%
SGSR-2 (ours)	budget 3.25	3.28	+45.4	3.29	+69.6
	budget 3.5	3.49	+27.0	3.40	+25.5
	budget 4.0	3.83	**+14.3**	3.62	**+18.7**
	budget 4.5	4.44	**+4.65**	4.55	**+3.97**
Uniform MLX + SQ	3-bit, k=128	3.28	+45.4	—	—
	3-bit, k=64	3.50	+30.9	3.50	+48.3
	3-bit, k=32	3.94	+23.1	3.93	+36.0
	4-bit, k=128	4.28	+6.45	—	—
	4-bit, k=64	4.50	+4.75	—	—
	4-bit, k=32	4.94	+3.42	—	—
	5-bit, k=64	5.50	+0.94	—	—
llama.cpp K-quants	`Q3_K_M`	3.99	+7.47	4.00	+7.79
	`Q4_K_M`	4.86	+2.97	4.92	+1.98
	`Q5_K_M`	5.69	+0.80	5.72	+0.66

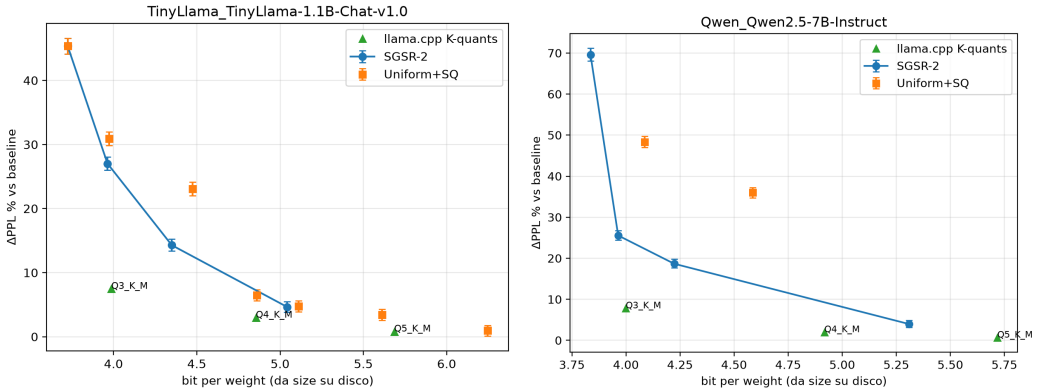

Figure 1: PPL-vs-bits frontiers for TinyLlama-1.1B (left) and Qwen2.5-7B (right). Error bars are 95% bootstrap CIs on per-token NLLs.

the best measured uniform setting yields +36.0% at *more* bits (3.93); at 3.40 bit/w SGSR-2 roughly halves the uniform degradation (+25.5% vs. +48.3% at 3.50). On TinyLlama the pattern is identical (+14.3% at 3.83 vs. +23.1% at 3.94). At the budget floor the allocator correctly degenerates to the uniform minimum configuration (identical numbers at 3.28 bit/w), and above ∼4.5 bit/w the two families converge as all configurations become quality-saturated.

SGSR-2 versus K-quants. The comparison splits cleanly at ∼4.4 bit/w. Above it, SGSR-2 sits on or below the interpolated K-quants frontier on both models: on Qwen, +3.97% at 4.55 bit/w against an interpolated ≈ + 4.3%; on TinyLlama, +4.65% at 4.44 against ≈ + 5.2%. Below 4 bit/w, `Q3_K_M` is clearly stronger (+7.8% at 4.00 vs. our +18.7% at 3.62 on Qwen). We attribute this gap to the storage format rather than the allocation: K-quants use super-blocks with 6-bit sub-scales and per-sub-block minima, a substantially richer low-bit representation than MLX's affine format (bf16 scale/bias per group), which is the space SGSR-2 allocates over. Where format richness is not the bottleneck (≥4.4 bit/w), measured allocation matches or beats hand-tuned heuristics; where it is, no allocation policy over the poorer format closes the gap. Extending MLX with a super-block

Table 3: Controlled re-evaluation of SGSR v1's entropy ranking (all variants 4-bit with SmoothQuant; v1's 100-sample evaluation protocol, retained here for comparability with v1; effective bits within ± 0.01 across rankings). Entropy is inside the random spread on both models; inverted ranking matches the correct one.

Ranking / baseline	**TinyLlama** ΔPPL%	**Qwen2.5-7B** ΔPPL%
Uniform k=64 + SQ (control)	**+3.00**	**+10.47**
Entropy tiers (v1)	+3.28	+10.88
Random tiers (3 seeds / 1 seed)	+3.26 – +3.93	+10.84
Inverted entropy tiers	+3.34	+11.00

format—and letting the same cost-table machinery allocate over it—is the natural next step, and MLX's recently added `mxfp4` mode (shared 8-bit scale per 32-group) is a first candidate.

5 Negative result: entropy ranking is indistinguishable from random

SGSR v1 reported that entropy-ranked group-size redistribution improved on uniform 4-bit quantization ($+3.28\%$ vs. $+4.34\%$ on TinyLlama). Two controls, absent in v1, overturn this conclusion (Table 3): (i) applying SmoothQuant to the uniform baseline—v1 applied it only to the treatment arm—already brings uniform to $+3.00\%$, *ahead* of v1's SGSR at equal effective bits; (ii) the entropy ranking performs within the spread of random rankings on both models, and an *inverted* entropy ranking performs the same as the correct one. The v1 improvement is therefore explained by the SmoothQuant confound, and the entropy proxy carries no allocation signal at this granularity. We report this correction explicitly so that others do not build on the retracted claim.

6 Related work

Post-training quantization. GPTQ [Frantar et al., 2022] minimizes layer-wise reconstruction error with approximate second-order information; AWQ [Lin et al., 2023] protects salient channels identified by activation magnitudes; SpQR [Dettmers et al., 2023] isolates sparse outliers at higher precision; SmoothQuant [Xiao et al., 2022] rescales activation difficulty into weights and is our pre-processing step.

Sensitivity-guided mixed precision. ZeroQ [Cai et al., 2020] measures per-layer sensitivity via KL divergence and selects bit-widths along a Pareto frontier; HAWQ-v3 [Yao et al., 2021] solves bit allocation as integer programming under Hessian sensitivity. SGSR-2 differs in three respects: the allocation space is the joint (bit-width, group-size) grid with metadata overhead inside the budget constraint; the cost signal is measured output-KL under fake-quantization of the actual target format (no Hessian or gradient); and the additivity assumption underlying the decomposed solver is explicitly tested rather than assumed. EWQ [Anonymous, 2025] uses entropy for bit-width selection; our Section 5 suggests caution with entropy proxies absent randomized controls.

Deployment-grade formats. llama.cpp's K-quants [Gerganov and contributors, 2023] are hand-designed super-block formats with empirically chosen per-tensor mixes; they are the de-facto consumer standard and our external baseline. Our results position measured allocation as complementary to format design: allocation wins where formats are comparable, formats win below 4 bit/w.

7 Limitations

(i) The uniform Qwen sweep at 4–5 bit/w is incomplete (an interrupted run; the measured 3-bit points suffice for the low-budget comparison but the 4-bit uniform column on Qwen is absent). (ii) The K-quants frontier below 4 bit/w (`Q2_K`, `Q3_K_S`) was not sampled. (iii) MLX and llama.cpp perplexity protocols differ in windowing; we mitigate by reporting deltas against each runtime's own full-precision baseline. (iv) Evaluation is perplexity-only; downstream tasks are future work. (v) One calibration seed on Qwen (three on TinyLlama's negative-result arm); the cost-table protocol fixes

the calibration set by construction. (vi) Two model families; broader architecture coverage would strengthen the claims.

8 Conclusion

Measured per-block cost tables, an explicitly validated additivity assumption, and an exact Lagrangian allocator yield the full quality/compression frontier of joint (bit-width, group-size) quantization from a single overnight profiling run on consumer hardware. Under unified accounting the method strictly dominates uniform MLX quantization on both tested models and matches or beats the hand-tuned K-quants frontier at $\geq$4.4 bit/w; the remaining low-bit gap is a storage-format limitation, not an allocation one. Equally important, controlled baselines overturned our own earlier entropy-based claim—we believe publishing such corrections, with the controls that produced them, is part of the contribution.

Acknowledgments

The author thanks Claude (Anthropic) for assistance with implementation, experimental orchestration, manuscript writing, and LaTeX typesetting. All research direction and final decisions are the author's.

References

Anonymous. Universality of layer-level entropy-weighted quantization beyond model architecture and size. *arXiv preprint arXiv:2503.04704*, 2025.

Apple Machine Learning Research. MLX: An array framework for apple silicon. `https://github.com/ml-explore/mlx`, 2023.

Yaohui Cai, Zhewei Yao, Zhen Dong, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. ZeroQ: A novel zero shot quantization framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.

Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*, 2023.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Georgi Gerganov and contributors. llama.cpp: Llm inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2023.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.

Zhewei Yao, Zhen Dong, Zhangcheng Zheng, Amir Gholami, Jiali Yu, Eric Tan, Leyuan Wang, Qijing Huang, Yida Wang, Michael W. Mahoney, and Kurt Keutzer. HAWQ-V3: Dyadic neural network quantization. In *International Conference on Machine Learning (ICML)*, 2021.

Peiyuan Zhang, Guangtai Zeng, Tianduo Wang, and Wei Lu. TinyLlama: An open-source small language model. In *arXiv preprint arXiv:2401.02385*, 2024.